

ASHISHKUMAR VERMA

AIML Engineer

+91 6354107144 | vermaaashish20@gmail.com | vermaaashish20.github.io
github.com/vermaaashish20 | linkedin.com/in/vermaaashish20

Summary

AI/ML Engineer architecting end-to-end AI solutions across Agentic systems, LLM pipelines, Computer vision, ML model training, and MLOps, translating complex business requirements into scalable, production-ready AI systems. Proficient in Python, PyTorch, HuggingFace Transformers, LangChain/LangGraph, RAG, FastAPI, and Azure with hands-on experience building across the LLM ecosystem including fine-tuning, prompt engineering, and agentic systems across real-world applications.

Skills

- Languages:** Python, JavaScript, TypeScript
- AI/ML:** PyTorch, TensorFlow, Scikit-learn, HuggingFace Transformers, LangChain/LangGraph, RAG, MLOps, Reinforcement Learning, Computer Vision (YOLO), Model Context Protocol (MCP)
- LLM & Agents:** OpenAI, Fine-tuning, Prompt Engineering, Agentic Systems, n8n
- Web Development:** Next.js, FastAPI, Node.js
- Cloud & Infrastructure:** Docker, Kubernetes, Terraform, Microsoft Azure, Azure AI Foundry, MLFlow, Kafka, GitHub Actions, CircleCI, Databricks
- Databases:** MongoDB, PostgreSQL, MySQL, Redis, Vector DBs (Pinecone, ChromaDB, Qdrant)
- Data Analytics:** Matplotlib, Seaborn

Education

B.Tech in Computer Science Engineering with MLAI | CGPA (8.01) Sep 2021 - Dec 2025
P.P. Savani University

Experience

AIML Engineer

Alphabin Technology Consulting

Jan 2025 - Current

- Architected and deployed an AI microservice for automated test failure analysis on Azure Container Apps, featuring an online learning ML pipeline for adaptive failure categorization (60–70% accuracy) across bug types (actual bug, UI change, flaky, miscellaneous), reducing tester debugging time significantly by surfacing probable failure causes and replacing a manual pass/fail/flaky-only workflow with actionable insights.
- Built an error grouping and analytics pipeline using embeddings for semantic grouping of test failures by errors, visualized in interactive graphs filterable by time range (7/14/30 days), branch, and error type, enabling teams to pinpoint high-impact errors and unique failures at a glance.
- Integrated an LLM-powered summary and insights pipeline into CI/CD workflows, automatically posting AI-generated test run summaries and failure insights directly into GitHub PRs and commits, giving developers and testers contextual failure analysis without leaving their CI/CD environment.
- Architected and built the TestDino MCP server, enabling AI coding assistants (Claude Code, Cursor, Claude Desktop) to query test results, analyze failure patterns, manage test suites, and create or update manual tests in natural language, bridging developer code, AI agents, and test management into a single unified debugging workflow.

- Designed and built an AI agent module for a VS Code extension (TypeScript, LangGraph, OpenAI, Playwright) that converts natural language test cases into executable Playwright scripts with step-wise browser automation, vision-based interaction, and dynamic locator generation, cutting test boilerplate creation time from ~20 minutes to ~10 minutes for a 15-step test.
- Built the integration microservice (Node.js, TypeScript) with OAuth2 for one-click (3-step) connections across Asana, Linear, monday.com, Azure DevOps, and Slack, decoupling integrations from the monolithic API gateway, reducing main server load, and automating issue creation with full error context (logs, screenshots, attachments) eliminating manual copy-paste workflows for QA teams.
- Led SOC 2 and ISO 27001 compliance governance and built an automated disaster recovery and failover system on Azure (Terraform, Azure Front Door) with cross-region provisioning and remediation of security gaps.
- Solely designed and developed a full-stack Internal SaaS platform for Construction Company to compile 500-700+ pages Detailed Project Reports (DPRs), replacing a 1-2 week manual compilation process of Excel measurement sheets, engineering drawings, vendor PDFs, and images into a single editor with rich formatting, reorderable sections, template support, and one-click PDF export, integrated with an AI editor for in-context text editing and generation.

Projects

LF2 Reinforcement Learning Game Agent | github.com/vermaashish20/Al-lf2

- Developed an AI agent for Little Fighter 2, collected and labeled gameplay data to train a YOLOv8n model for real-time detection of characters, enemies, and weapons, powering a rule-based agent that uses detected positions for proximity-based targeting and attacks.
- Built a custom Gymnasium environment using game RAM memory addresses for direct hero and enemy state detection, modeling full character movesets and special moves, enabling a PPO-based RL agent trained up to 6000 steps to achieve autonomous enemy combat without the action delay inherent in the vision-based approach.

Zeno - YouTube RAG | github.com/vermaashish20/zeno

- Built a full-stack YouTube RAG platform (FastAPI, Next.js, LangGraph) that downloads and transcribes public YouTube videos and playlists using a quantized Whisper tiny model (16-bit) with semantic chunking, storing transcriptions in a ChromaDB vector workspace per playlist for conversational learning across entire playlists without watching the content.
- Implemented advanced retrieval features including timestamp citations linking answers back to source video moments, persistent chat memory per workspace, markdown workspace export for use with external AI tools, and multi-provider LLM support (Ollama, NVIDIA, Gemini).

Bookish – AI-Powered Multi-Agent Book Authoring Platform | github.com/vermaashish20/bookish

- Built a full-stack AI authoring platform using Next.js, TypeScript, and FastAPI, enabling real-time agent interaction and rich-text editing for long-form AI content generation, while integrating Langfuse for observability, RAG tracing, and multi-agent execution monitoring.
- Architected and developed a stateful multi-agent AI system using Python, FastAPI, and LangGraph, orchestrating Planner, Researcher, World Builder, Writer, Editor, Memory Keeper, and Fact-Checker agents through a DAG workflow to automate publication-grade long-form content generation (25K+ words)
- Engineered a memory-driven RAG pipeline by integrating MongoDB (episodic memory) and ChromaDB (vector database), enabling context-aware reasoning, factual grounding, and persistent multi-agent state management.